

Колледж Университета «Сириус»

ОТЧЁТ

по лабораторной работе № 5

**ШАРДИРОВАНИЕ POSTGRESQL: ROUTER И CONSISTENT
HASHING**

по модулю «Масштабирование баз данных»

Исполнитель:

обучающийся группы K0709-23/3

подпись, дата

У. И. Усачева

Руководитель:

преподаватель

подпись, дата

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1 Выбор данных для шардирования.....	6
2 Запуск нескольких PostgreSQL.....	8
3 Реализация Router	10
3.1 Хэш-функция.....	10
3.2 $\text{shard} = \text{hash}(\text{shard_key}) \% N$	10
3.3 Router внутри сервиса.....	11
4 Распределение данных.....	13
5 Добавление нового шарда при $\text{hash}(\text{key}) \% N$	15
6 Реализация Consistent Hashing.....	16
6.1 Как устроено кольцо	16
6.2 Распределение на кольце.....	17
7 Сравнение стратегий.....	18
7.1 $3 \rightarrow 4$ шарда	18
7.2 Виртуальные узлы.....	19
8 Проверка на реальном переносе данных	21
9 Ответы на вопросы для самопроверки.....	23
ЗАКЛЮЧЕНИЕ	26
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	27
ПРИЛОЖЕНИЕ А Router: $\text{hash}(\text{key}) \% N$ и hash ring.....	28
ПРИЛОЖЕНИЕ Б Расчёт распределения и переноса данных	31
ПРИЛОЖЕНИЕ В Загрузка и перенос данных между шардами	33

ВВЕДЕНИЕ

Реплика из лабораторной работы № 4 увеличивает число чтений, которое может обслужить система, но каждая реплика хранит полную копию данных, а запись по-прежнему идёт на один сервер. Когда данных становится больше, чем помещается на один сервер, или запись упирается в его ресурсы, данные приходится делить между несколькими независимыми серверами — шардировать.

Цель работы — распределить данные сервиса TaskManager между несколькими экземплярами PostgreSQL и понять, как выбор стратегии шардирования влияет на объём переносимых данных при изменении количества шардов.

Для достижения цели решались следующие задачи:

- выбрать сущность сервиса и shard key и обосновать выбор;
- запустить три независимых экземпляра PostgreSQL;
- реализовать в сервисе router со стратегией $\text{hash}(\text{key}) \% N$ и распределить задачи между шардами;
- посчитать, какая доля данных переедет при добавлении четвёртого шарда;
- реализовать consistent hashing с hash ring и виртуальными узлами и сравнить стратегии;
- проверить расчёт настоящим переносом данных на четвёртый PostgreSQL.

При выполнении использовались документация PostgreSQL и Npgsql [1–3], оригинальная работа о consistent hashing [4], описание виртуальных узлов в Amazon Dynamo [5], книга М. Клеппмана [6], методические указания [7] и инструмент записи терминала VHS [8].

Таблица 1 – Среда выполнения экспериментов

Параметр	Значение
СУБД	PostgreSQL 16.14 (postgres:16-alpine) в Docker Desktop, Windows 11

Параметр	Значение
Шарды	3 независимых экземпляра postgres-shard-0...2 (порты 5440–5442) и четвёртый postgres-shard-3 (5443) для опыта с добавлением шарда
Основная база	postgres (5432): пользователи, проекты, теги, исходные задачи и топология шардов
Сервис	TaskManager: .NET 8, EF Core 8, Dapper, Npgsql
Данные	1 000 000 задач сервиса, 10 000 пользователей

Все эксперименты автоматизированы:

Таблица 2 – Скрипты и результаты

Задание	Запуск	Результаты
2 — несколько PostgreSQL	bash scripts/lab-05/01-shards-status.sh	docs/lab-05/results/01-status.txt
3–5 — router hash % N, распределение, добавление шарда	bash scripts/lab-05/02-modulo.sh	02-modulo.txt, 02-modulo-plan.json
6–7 — consistent hashing и сравнение	bash scripts/lab-05/03-consistent-hashing.sh	03-consistent-hashing.txt, 03-ring-plan.json
Реальный перенос данных 3 → 4 → 3	bash scripts/lab-05/04-real-rebalance.sh	04-real-rebalance.txt
Router внутри сервиса	bash scripts/lab-05/05-api-routing.sh	05-api-routing.txt
Всё подряд	bash scripts/lab-05/run-all.sh	docs/lab-05/results/
Запись терминала	vhs scripts/lab-05/demo.tape	docs/lab-05/demo.gif

Как измерялось. Количество записей на шардах считается SQL-запросом прямо на каждом экземпляре PostgreSQL, мимо сервиса. Перенос при изменении числа шардов считает сервис: для каждого пользователя он определяет шард до и после изменения и суммирует задачи тех, у кого шард поменялся. Прогноз проверен настоящим переносом данных на четвёртый PostgreSQL.

Запись терминала. GIF снят инструментом VHS на работающем сервисе: задачи на трёх шардах, решение router'a для одного пользователя и сравнение стратегий при добавлении и удалении шарда.

```

> powershell -ExecutionPolicy Bypass -File scripts/lab-05/demo-sharding.ps1 -Pause 3

=== Три независимых PostgreSQL: задачи на каждом шарде (SQL мимо сервиса)
Shard 0: 339959 задач, 3480 пользователей
Shard 1: 319756 задач, 3130 пользователей
Shard 2: 340285 задач, 3390 пользователей

=== Router: куда попадают задачи пользователя gen_user_1
user_id = 3592598a-0d19-41d9-b5fb-77b485b62b20
hash = 8455333448350808900
hash % 3 -> Shard 2 hash % 4 -> Shard 0
ring N=3 -> Shard 1 ring N=4 -> Shard 1

=== Добавляем шард: 3 -> 4. Сколько задач меняет shard
Стратегия      Перемещено  Пользователей  Куда уходят задачи
Modulo          75,34 %      7516           на шарды 0, 1, 2, 3
ConsistentHashing 27,35 %      2668           на шарды 3

=== Удаляем шард: 4 -> 3
Modulo          75,34 %      задачи уходят с шардов 0, 1, 2, 3
ConsistentHashing 27,35 %      задачи уходят с шардов 3
>

```

Рисунок 1 – Три шарда, router для пользователя gen_user_1 и перенос задач при $3 \rightarrow 4$ и $4 \rightarrow 3$ для $\text{hash \% N}$ и consistent hashing

1 Выбор данных для шардирования

1) **Какую сущность вы выбрали?** Задачи — таблицу `tasks`. Это самая большая и быстрее всех растущая таблица сервиса: миллион строк против 10 тысяч пользователей и тысячи проектов. Именно её объём упирается в возможности одного сервера, остальные таблицы маленькие и остаются в основной базе.

2) **Какое поле используется как `shard key`?** `user_id` — владелец задачи.

3) Почему выбран именно этот ключ?

- почти все запросы сервиса к задачам выполняются в контексте пользователя: список своих задач, создание, статистика, проверка прав;

- все задачи одного пользователя оказываются на одном шарде, поэтому его список читается одним запросом к одному серверу, без сбора данных со всех шардов;

- ключ не меняется: задача не переходит к другому владельцу, значит запись никогда не переезжает из-за изменения ключа;

- `user_id` — UUID, после хэширования он даёт равномерно распределённые значения.

Отклонённые варианты: `id` задачи распределил бы строки идеально, но список задач пользователя пришлось бы собирать со всех шардов; `project_id` может быть пустым; `created_at` сложил бы всю свежую нагрузку на один шард.

4) **Насколько равномерно он будет распределять данные?** По числу пользователей — практически равномерно: хэш от UUID ведёт себя как случайное число. По числу задач — хуже, потому что пользователи неодинаковы: медиана 72 задачи на пользователя, 99-й перцентиль — 502, а у самого активного `gen_user_1` — 9 900 задач, почти 1 % всей таблицы. Такого пользователя нельзя разрезать между шардами, он целиком лежит на одном — это «горячий ключ».

5) Какие запросы сервиса используют этот ключ?

Таблица 3 – Запросы сервиса и shard key

Запрос	Использует user_id	На сколько шардов идёт
GET /api/tasks пользователя — список своих задач	да, WHERE user_id = ...	1
POST /api/tasks — создание задачи	да, владелец известен из токена	1
GET /api/tasks/stats для пользователя	да	1
Проверка владельца задачи (IsOwnerAsync)	да, если известен владелец	1
GET /api/tasks/{id} без владельца	нет	все шарды
Список всех задач администратора, статистика по всем	нет	все шарды

2 Запуск нескольких PostgreSQL

Шарды описаны в `docker-compose.yml` через общий шаблон: одинаковый образ, база `taskmanager_shard` и схема, которая создаётся при первом запуске контейнера из `docker/postgres/shard-init.sql`.

Листинг 1 – Шаблон шарда и шарды в `docker-compose.yml`

```
x-postgres-shard: &postgres-shard
  image: postgres:16-alpine
  environment:
    POSTGRES_USER: taskmanager
    POSTGRES_PASSWORD: taskmanager_password
    POSTGRES_DB: taskmanager_shard
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U taskmanager -d taskmanager_shard"]

services:
  postgres-shard-0:
    <<: *postgres-shard
    container_name: taskmanager-shard-0
    ports:
      - "5440:5432"
    volumes:
      - shard_0_data:/var/lib/postgresql/data
      - ./docker/postgres/shard-init.sql:/docker-entrypoint-initdb.d/01-shard.sql:ro

# postgres-shard-1 (5441) и postgres-shard-2 (5442) – так же

postgres-shard-3:
  <<: *postgres-shard
  container_name: taskmanager-shard-3
  profiles: ["scale"] # запускается отдельно для опыта «3 → 4 шарда»
  ports:
    - "5443:5432"
```

Таблица 4 – Экземпляры PostgreSQL проекта

Экземпляр	Контейнер	Порт на хосте	Что хранит
Основная база	taskmanager-postgres	5432	пользователи, проекты, теги, топология шардов
Shard 0	taskmanager-shard-0	5440	задачи пользователей, попавших на шард 0
Shard 1	taskmanager-shard-1	5441	задачи шарда 1
Shard 2	taskmanager-shard-2	5442	задачи шарда 2
Shard 3	taskmanager-shard-3	5443	новый шард для опыта с добавлением, профиль scale

Шарды — действительно независимые серверы, а не базы внутри одного: у каждого свой кластер с уникальным `system_identifier`, свой каталог данных и время запуска.

Листинг 2 – Три независимых кластера PostgreSQL

shard	database	started	system_identifier
shard-0	taskmanager_shard	2026-09-15 15:19:37.785006+00	7685782026353578018
shard-1	taskmanager_shard	2026-09-15 15:19:37.797762+00	7685782026359484450
shard-2	taskmanager_shard	2026-09-15 15:19:37.786107+00	7685782026356277282

Схема таблицы одинакова на всех шардах. Ключ шардирования входит в первичный ключ (`user_id`, `id`). Внешних ключей на пользователей и проекты нет: они лежат в другой базе, и PostgreSQL не может проверять целостность между разными серверами — теперь это ответственность сервиса.

Листинг 3 – Схема таблицы tasks на шарде

Column	Type	Nullable
id	uuid	not null
user_id	uuid	not null
project_id	uuid	
title	character varying(300)	not null
description	text	
status	integer	not null
priority	integer	not null
due_date	timestamp with time zone	
created_at	timestamp with time zone	not null
updated_at	timestamp with time zone	not null
Indexes:		
"tasks_pkey" PRIMARY KEY, btree (user_id, id)		
"idx_tasks_user_id created_at" btree (user_id, created_at DESC)		

Строки подключения ко всем шардам сервис получает из конфигурации (`Sharding:Shards`). А стратегия `router'a` и число используемых шардов хранятся в основной базе, в таблице `shard_topology` (миграция 014). Так после переноса данных все экземпляры сервиса начинают маршрутизировать одинаково, в том числе после перезапуска.

3 Реализация Router

3.1 Хэш-функция

Router должен давать один и тот же ответ в любом процессе и на любой машине. Стандартный `GetHashCode()` в .NET для этого не годится: он случайный для каждого запуска процесса. Поэтому используется MD5 от строкового вида `user_id`, а первые 8 байт дайджеста дают 64-битное число:

Листинг 4 – Стабильный хэш ключа

```
public static ulong Of(string key)
{
    Span<byte> digest = stackalloc byte[MD5.HashSizeInBytes];
    MD5.HashData(Encoding.UTF8.GetBytes(key), digest);
    return BitConverter.ToUInt64(digest[..8]);
}
```

MD5 здесь не защищает данные, а только перемешивает биты, поэтому его криптографическая слабость не имеет значения.

3.2 $\text{shard} = \text{hash}(\text{shard_key}) \% N$

Листинг 5 – Router $\text{hash}(\text{key}) \% N$

```
public sealed class ModuloShardRouter : IShardRouter
{
    public ModuloShardRouter(int shardCount) => ShardCount = shardCount;

    public ShardStrategy Strategy => ShardStrategy.Modulo;

    public int ShardCount { get; }

    public int GetShard(Guid key) => (int)(ShardHash.Of(key) % (ulong)ShardCount);
}
```

Таблица 5 – Решения router'a для пользователей сервиса

Пользователь	user_id	hash(user_id)	% 3	% 4
gen admin	91f85bfc-...737	5775129772183095613	Shard 1	Shard 1
gen user 1	3592598a-...b20	8455333448350808900	Shard 2	Shard 0
gen user 2	334e7749-...969	17822400836141746365	Shard 0	Shard 1
gen user 3	463b8fdf-...603	15299553839507782216	Shard 1	Shard 0
gen user 100	462b8287-...776	6138247566949930649	Shard 2	Shard 1

3.3 Router внутри сервиса

Таблица 6 – Компоненты шардирования в сервисе

Компонент	Слой	Роль
ShardHash	Application	Стабильный 64-битный хэш ключа
ModuloShardRouter, ConsistentHashRouter	Application	Две стратегии router'a за общим интерфейсом IShardRouter
ShardPlanner	Application	Распределение ключей по шардам и расчёт переноса при изменении их числа
ShardingService	Application	Выбирает шард для операций, загружает данные, переносит пользователей
ShardedTaskStore	Infrastructure	Подключения к шардам, COPY при загрузке и переносе, чтение и запись задач
ShardsController	API	/api/shards/..., роль Admin

Таблица 7 – Endpoint'ы шардирования

Endpoint	Что делает
GET /api/shards	Топология и реальное количество задач и пользователей на каждом шарде
GET /api/shards/route/{userId}	Хэш ключа и его шард для $\text{hash} \% N$ и hash ring при разном N
GET /api/shards/users/{userId}/tasks	Задачи пользователя с того шарда, который выбрал router
POST /api/shards/users/{userId}/tasks	Создание задачи на шарде пользователя
POST /api/shards/load?strategy=&shards=&virtualNodes=	Загрузка всех задач основной базы на шарды
GET /api/shards/simulate/distribution, .../simulate/rebalance	Распределение и перенос для другой топологии, без изменения данных
POST /api/shards/rebalance?shards=	Настоящий перенос данных при изменении числа шардов

Проверка на работающем сервисе: задача создаётся через API, router выбирает шард, а SQL на всех трёх экземплярах показывает, что строка лежит только на выбранном.

Листинг 6 – Задача, созданная через router, и её поиск на всех шардах

```
POST /api/shards/users/462b8287-da8e-4d42-9496-8c0a9653c776/tasks
{ "shard": 2, "foundOnShards": [2], "task": { "title": "LR5: task routed by user_id", ... } }

shard-0 | 0
shard-1 | 0
```

Чтение задач того же пользователя идёт на один шард и обслуживается индексом:

Листинг 7 – План запроса задач пользователя на его шарде

```
Limit (actual time=0.043..0.053 rows=10 loops=1)
->  Index Scan using idx_tasks_user_id_created_at on tasks (actual time=0.042..0.052 rows=10
loops=1)
      Index Cond: (user_id = '462b8287-da8e-4d42-9496-8c0a9653c776'::uuid)
Execution Time: 0.122 ms
```

4 Распределение данных

В сервис загружены все задачи основной базы — 1 000 000 штук. Сервис одним потоковым проходом читает таблицу `tasks` и отправляет каждую строку в `COPY`-поток того шарда, который выбрал `router`. Загрузка заняла 5,4 секунды.

Таблица 8 – Распределение задач при `shard = hash(user_id) % 3` (SQL на шардах)

Шард	Задач	Пользователей	Доля задач	Размер таблицы
Shard 0	326 625	3 314	32,66 %	82 МБ
Shard 1	335 513	3 295	33,55 %	84 МБ
Shard 2	337 862	3 391	33,79 %	85 МБ
Всего	1 000 000	10 000	100 %	251 МБ

Самый большой шард больше среднего в 1,014 раза, относительное отклонение размеров — 1,45 %.

Таблица 9 – Самые тяжёлые пользователи каждого шарда

Шард	Пользователь	Задач
Shard 0	<code>gen_user_2</code>	4 160
Shard 1	<code>gen_user_3</code>	3 136
Shard 2	<code>gen_user_1</code>	9 900

Выводы.

– **Насколько равномерно.** Практически идеально: доли шардов 32,7–33,8 % при идеале 33,3 %, разница между крайними шардами 1,1 процентного пункта. Хэш от `UUID` равномерен, а 10 000 ключей достаточно, чтобы случайные отклонения сгладились.

– **Есть ли заметный дисбаланс.** По записям — нет. Но дисбаланс сидит внутри: самый активный пользователь `gen_user_1` целиком лежит на `Shard 2` и один даёт ему 9 900 задач. Сейчас это 3 % шарда, и общую картину он не портит.

– **Может ли дисбаланс стать проблемой.** Да, в двух случаях. Во-первых, если один пользователь (или организация) вырастет до заметной доли всех данных: его нельзя разрезать, и шард, на котором он живёт, будет больше и нагруженнее остальных, сколько бы шардов мы ни добавили. Во-

вторых, равное число строк не означает равную нагрузку: активный пользователь делает больше запросов, поэтому по CPU шард может быть перегружен даже при одинаковом объёме данных.

5 Добавление нового шарда при $\text{hash}(\text{key}) \% N$

К системе добавляется четвёртый экземпляр: вместо $\text{hash}(\text{key}) \% 3$ теперь $\text{hash}(\text{key}) \% 4$. Для каждого из 10 000 пользователей сервис вычислил шард до и после изменения и сложил задачи тех, у кого шард поменялся.

Таблица 10 – Переход $\text{hash}(\text{key}) \% 3 \rightarrow \text{hash}(\text{key}) \% 4$

Показатель	Значение
Всего записей	1 000 000
Новый шард	Shard 3
Изменили shard	753 446
Не изменили	246 554
Перемещено	75,34 %
Пользователей переехало	7 516 из 10 000

Таблица 11 – Куда уходят задачи при $\text{hash} \% 3 \rightarrow \text{hash} \% 4$

Откуда \ Куда	Shard 0	Shard 1	Shard 2	Shard 3
Shard 0	—	83 919	80 820	82 350
Shard 1	86 879	—	85 954	78 915
Shard 2	93 824	77 682	—	83 103

Почему добавление одного PostgreSQL требует перемещения большого объёма данных. Номер шарда — это остаток от деления хэша на N , и при смене N остаток меняется почти у всех ключей. Ключ остаётся на месте, только если $\text{hash} \% 3 == \text{hash} \% 4$, то есть если $\text{hash} \% 12$ равен 0, 1 или 2 — это 3 случая из 12, ровно 25 %. Остальные 75 % ключей переезжают, что и показал замер: 75,34 %.

Главное — матрица переносов. Данные не просто уезжают на новый шард: старые шарды массово обмениваются задачами между собой ($0 \rightarrow 1$, $1 \rightarrow 2$, $2 \rightarrow 0 \dots$), хотя на них ничего не изменилось. Новому шарду нужна четверть данных, а переезжают три четверти. На время переноса это нагрузка на все серверы сразу и долгое окно, когда часть задач уже переехала, а часть ещё нет.

6 Реализация Consistent Hashing

6.1 Как устроено кольцо

Хэши занимают диапазон от 0 до $2^{64} - 1$; замкнём его в кольцо, где после максимума снова идёт 0.

1) Каждый шард ставит на кольцо свои точки — **виртуальные узлы**. Точка — хэш строки `shard-{номер}#vnode-{номер узла}`, поэтому её положение зависит только от номера шарда и узла и не зависит от того, сколько всего шардов.

2) Ключ хэшируется в точку на том же кольце.

3) Ключ принадлежит шарду, чья точка первая по часовой стрелке от точки ключа. Если дальше точек нет, поиск продолжается с начала кольца.

Точки хранятся в отсортированном массиве, поэтому поиск двоичный: около 11 сравнений для трёх шардов по 500 узлов.

Листинг 8 – Hash ring с виртуальными узлами

```
public ConsistentHashRouter(int shardCount, int virtualNodes = 100)
{
    var ring = new List<(ulong Point, int Shard)>(shardCount * virtualNodes);
    for (var shard = 0; shard < shardCount; shard++)
    {
        for (var node = 0; node < virtualNodes; node++)
        {
            ring.Add((ShardHash.Of($"shard-{shard}#vnode-{node}"), shard));
        }
    }

    ring.Sort((a, b) => a.Point.CompareTo(b.Point));
    _points = ring.Select(r => r.Point).ToArray();
    _owners = ring.Select(r => r.Shard).ToArray();
}

public int GetShard(Guid key)
{
    var index = Array.BinarySearch(_points, ShardHash.Of(key));
    if (index < 0)
    {
        index = ~index;           // первая точка больше хэша ключа
    }

    return _owners[index == _points.Length ? 0 : index]; // за последней точкой — снова
    начало кольца
}
```

Главное свойство следует из первого пункта: кольцо для $N + 1$ шардов — это кольцо для N шардов плюс точки нового шарда. Точки старых шардов

остаются на местах, поэтому ключ может сменить шард только в одном случае — если между ним и его прежней точкой встала точка нового шарда.

6.2 Распределение на кольце

Таблица 12 – Распределение задач через hash ring, 3 шарда, 100 виртуальных узлов (SQL на шардах)

Шард	Задач	Пользователей	Доля задач
Shard 0	304 908	3 026	30,49 %
Shard 1	303 844	2 922	30,38 %
Shard 2	391 248	4 052	39,12 %

Кольцо со 100 виртуальными узлами распределило данные заметно хуже, чем $\text{hash} \% 3$: Shard 2 получил 39 % задач, в 1,174 раза больше среднего. Дело не в тяжёлых пользователях — перекос такой же по их числу (4 052 против 2 922), — а в том, как случайно легли точки: дуги кольца, доставшиеся шарду 2, в сумме оказались длиннее. Как это печат виртуальные узлы — в разделе 7.

7 Сравнение стратегий

7.1 3 → 4 шарда

Таблица 13 – Перенос данных при добавлении четвёртого шарда

Стратегия	Перемещено задач	Перемещено пользователей	Куда уходят задачи
$\text{hash}(\text{key}) \% N$	75,34 % (753 446)	7 516	на все шарды, включая старые
Consistent Hashing, 100 узлов	24,09 % (240 903)	2 444	только на Shard 3
Consistent Hashing, 500 узлов	27,35 % (273 461)	2 668	только на Shard 3

Таблица 14 – Потоки задач при 3 → 4 на hash ring (100 узлов)

Откуда	Куда	Задач
Shard 0	Shard 3	76 922
Shard 1	Shard 3	68 713
Shard 2	Shard 3	95 268

1. Почему результаты отличаются. В $\text{hash} \% N$ число шардов входит в формулу для каждого ключа, поэтому при изменении N расположение пересчитывается у всех ключей и совпадает случайно лишь у четверти. В consistent hashing ключ привязан не к N , а к ближайшей точке на кольце. Добавление шарда добавляет точки, но не сдвигает существующие, поэтому у большинства ключей ближайшая точка остаётся прежней.

2. Почему consistent hashing уменьшает объём переноса. Переезжают только ключи, попавшие на дуги перед новыми точками, и все они едут на новый шард. Обмена между старыми шардами нет. Новому шарду нужна примерно $1/(N + 1)$ данных — при 3 → 4 это 25 %, — и примерно столько и переносится: 24,1 % при 100 узлах, 27,4 % при 500. Точное значение зависит от того, какую долю кольца получил новый шард.

3. Что произойдёт, если добавить ещё один шард.

Таблица 15 – Последовательное добавление шардов

Изменение	$\text{hash} \% N$	Consistent Hashing, 100 узлов	Доля, нужная новому шарду
3 → 4	75,34 %	24,09 %	25,0 %
4 → 5	79,41 %	20,06 %	20,0 %

Изменение	hash % N	Consistent Hashing, 100 узлов	Доля, нужная новому шарду
5 → 6	83,90 %	13,77 %	16,7 %

С ростом числа шардов hash % N становится только хуже: на месте остаются лишь ключи с совпадающими остатками, их доля стремится к $1/(N + 1)$. Consistent hashing, наоборот, переносит всё меньшую долю — примерно столько, сколько забирает себе новый шард.

4. Что произойдёт, если удалить шард.

Таблица 16 – Удаление шарда

Изменение	hash % N	Consistent Hashing, 100 узлов
4 → 3	75,34 %	24,09 % — только данные удалённого Shard 3
3 → 2	67,70 %	39,12 % — ровно содержимое Shard 2

При удалении шарда из кольца исчезают его точки. Его ключи достаются следующим по кольцу точкам разных оставшихся шардов (76 922 задачи уходят на Shard 0, 68 713 — на Shard 1, 95 268 — на Shard 2), а ключи остальных шардов не двигаются вовсе. hash % N снова перетасовывает почти всё. Удаление Shard 2 из трёх переносит в кольцо 39 % — это не недостаток алгоритма, а размер удаляемого шарда: переносится ровно то, что на нём лежало.

7.2 Виртуальные узлы

Таблица 17 – Влияние числа виртуальных узлов (3 шарда)

Узлов на шард	Самый большой / средний	Отклонение размеров	Перемещено при 3 → 4
1	2,451	103,13 %	77,48 %
10	1,444	39,88 %	32,51 %
50	1,057	7,01 %	25,45 %
100	1,174	12,29 %	24,09 %
500	1,021	2,88 %	27,35 %
1000	1,066	4,95 %	26,40 %
для сравнения: hash % 3	1,014	1,45 %	75,34 %

С одним узлом на шард кольцо почти бесполезно: три точки делят кольцо на три случайные дуги, и один шард получил в 2,45 раза больше среднего. С ростом числа узлов доля шарда складывается из многих

маленьких дуг, и сумма приближается к равной. Зависимость не строго монотонная — 100 узлов в этом замере случайно оказались хуже 50, — но порядок ясен: десятки узлов дают перекося в десятки процентов, сотни — единицы процентов. Для сервиса выбрано 500 узлов: перекося около 2 % при переносе, близком к идеальным 25 %.

8 Проверка на реальном переносе данных

Прогноз проверен на живых данных. Сервис переведён на hash ring с 500 узлами, запущен четвёртый PostgreSQL, и сервис перенёс задачи командой `POST /api/shards/rebalance?shards=4`.

Перенос идёт пачками по 500 пользователей. Задачи копируются на новый шард через `COPY` во временную таблицу и `INSERT ... ON CONFLICT DO NOTHING`, и только после фиксации копии удаляются с исходного шарда. Общей транзакции у двух баз нет, поэтому порядок шагов выбран так, чтобы сбой в середине оставил дубликат, но не потерю; повторный запуск дубликат уберёт. Топология в основной базе переключается только после того, как данные на месте.

Таблица 18 – Прогноз и настоящий перенос 3 → 4 → 3 (hash ring, 500 узлов)

Шаг	Прогноз	Перенесено на самом деле	Время
3 → 4, запуск Shard 3	273 461 задача, 2 668 пользователей	273 461 задача, 2 668 пользователей	8,3 с
4 → 3, удаление Shard 3	273 461 задача	273 461 задача, на Shard 3 осталось 0	8,1 с

Таблица 19 – Задачи на шардах до и после переноса (SQL на шардах)

Шард	3 шарда	После 3 → 4	После 4 → 3
Shard 0	339 959	236 505	339 959
Shard 1	319 756	244 860	319 756
Shard 2	340 285	245 174	340 285
Shard 3	—	273 461	0

Прогноз совпал с реальностью до задачи. Пользователь `gen_user_3` до переноса читался с Shard 0, после — с Shard 3, после удаления шарда — снова с Shard 0, и во всех трёх случаях сервис вернул все его 3 136 задач.

Побочный эффект переноса виден по размеру таблиц: после удаления и обратного копирования строк шарды выросли с 85 до 98 МБ. `DELETE` в PostgreSQL не освобождает место сразу, а оставляет мёртвые версии строк до `VACUUM`, поэтому после массового переноса шарды нужно вакуумировать.

Ограничение решения: пока идёт перенос, топология ещё старая, и задачи уже переехавшего пользователя читаются со старого шарда, где их

уже нет. В промышленной системе на это время нужны чтение с двух шардов или запрет записи для переносимых пользователей.

9 Ответы на вопросы для самопроверки

1) **Что такое shard?** Отдельный экземпляр базы данных — со своими ресурсами сервера, диском и процессами, — который хранит часть данных общей логической таблицы. В работе это три контейнера `postgres-shard-0...2`, каждый с собственным кластером PostgreSQL и своей долей задач.

2) **Что такое shard key?** Поле записи, по которому router определяет её шард. От него зависят равномерность распределения, какие запросы идут на один шард, а какие на все, и придётся ли переносить запись при изменении данных. В работе это `user_id`.

3) **Почему нельзя просто случайно распределять записи между PostgreSQL?** Потому что запись потом нужно найти. При случайном размещении шард записи нигде не зафиксирован, и любой запрос — даже чтение одной задачи — пришлось бы отправлять на все шарды. Либо пришлось бы вести отдельную таблицу «запись → шард», которая сама становится узким местом. Кроме того, связанные данные — все задачи одного пользователя — разлетелись бы по разным серверам. Детерминированная функция от shard key позволяет вычислить шард, ничего не спрашивая у базы.

4) **Как работает `hash(key) % N`?** Хэш-функция превращает ключ в большое число — в работе это первые 8 байт MD5 от `user_id`. Остаток от деления этого числа на количество шардов `N` даёт номер шарда от 0 до `N - 1`. Хорошая хэш-функция делает остатки равновероятными, поэтому данные делятся почти поровну: 32,66 / 33,55 / 33,79 %.

5) **Почему изменение количества shard'ов является проблемой?** Потому что `N` входит в формулу каждого ключа. После смены `N` у большинства ключей остаток становится другим, и их данные лежат не там, где их будет искать router. Перед переключением эти данные нужно перенести: при `3 → 4` это 75 % задач (753 446 из миллиона), при `5 → 6` — уже

84 %. Перенос нагружает все шарды, занимает время и создаёт окно, когда часть данных уже на новом месте, а часть ещё на старом.

6) **Что такое Consistent Hash Ring?** Способ маршрутизации, при котором пространство хэшей замыкается в кольцо. Каждый шард ставит на кольцо свои точки, ключ хэшируется в точку на том же кольце и принадлежит шарду, чья точка ближайшая по часовой стрелке. В реализации сервиса поиск шарда — двоичный поиск по отсортированному массиву точек.

7) **Почему при добавлении shard'a не приходится переносить все данные?** Положение точек существующих шардов не зависит от числа шардов, поэтому при добавлении нового они не сдвигаются. Новый шард лишь добавляет свои точки, и шард меняется только у ключей, для которых новая точка оказалась ближе прежней. Все такие ключи переезжают на новый шард — примерно $1/(N + 1)$ данных. В работе при $3 \rightarrow 4$ это 24–27 %, и ни одна задача не переехала между старыми шардами.

8) **Что такое virtual nodes и зачем они нужны?** Виртуальные узлы — несколько точек на кольце для одного физического шарда. С одной точкой на шард кольцо делится на несколько случайных дуг разной длины, и шарды получают очень разные доли: в замере один из трёх шардов получил в 2,45 раза больше среднего. Когда у шарда сотни точек, его доля складывается из многих маленьких дуг и приближается к равной: при 500 узлах перекос около 2 %. Ещё виртуальные узлы делают перенос мягче: новый шард забирает понемногу у всех, а данные удалённого шарда расходятся по всем оставшимся, а не достаются одному соседу. А меняя число узлов, можно дать более мощному серверу больше данных.

9) **Может ли Consistent Hashing гарантировать идеальное распределение нагрузки?** Нет. Во-первых, даже по числу ключей распределение только приблизительно равное и зависит от случайного положения точек: 100 узлов в работе дали 39 % на одном шарде из трёх. Во-вторых, алгоритм распределяет ключи, а не нагрузку: пользователь с 9 900

задачами и пользователь с одной задачей для него одинаковы, и «горячий» ключ целиком лежит на одном шарде. В-третьих, нагрузка определяется не только объёмом данных, но и частотой запросов. Consistent hashing решает проблему переноса данных, а не проблему равномерности.

10) Что произойдёт, если один shard станет значительно более загруженным остальных? Он станет узким местом всей системы: запросы пользователей этого шарда будут медленнее, а при исчерпании диска или CPU перестанут обслуживаться, хотя остальные шарды простаивают. Простое добавление шарда помогает лишь частично: новый шард заберёт часть ключей, и не обязательно у перегруженного. Решение зависит от причины. Если неудачно легли точки — поменять число виртуальных узлов. Если виноват один горячий ключ — вынести его на выделенный шард или сделать ключ составным, например `user_id` и проект. Если перегрузка по чтению — добавить этому шарду реплики, как в ЛР № 4. Во всех случаях перекос нужно замечать заранее, поэтому размер и нагрузку шардов надо мониторить так же, как отставание реплики.

ЗАКЛЮЧЕНИЕ

Shard — это отдельный экземпляр базы данных, который хранит часть общих данных: у каждого шарда своё подмножество строк, а вместе шарды содержат всё. Shard key — поле, по которому определяется, на каком шарде лежит запись; в TaskManager это `user_id`, поэтому все задачи пользователя живут на одном шарде и читаются одним запросом. Router — компонент сервиса, который по shard key решает, к какому PostgreSQL обратиться: без него приложение не знало бы, где искать запись, и опрашивало бы все шарды. Стратегия $\text{hash}(\text{key}) \% N$ берёт остаток от деления хэша ключа на число шардов; она проста и распределяет данные почти идеально — 32,7–33,8 % на трёх шардах. Её проблема — изменение N : остаток пересчитывается у всех ключей, и при переходе с трёх шардов на четыре переезжают 75 % задач, причём старые шарды массово обмениваются данными между собой. Consistent hashing размещает шарды точками на кольце хэшей, и новый шард лишь добавляет свои точки, не сдвигая чужие: переезжает только то, что достаётся новому шарду, — 24–27 % вместо 75 %, и всё это идёт на новый шард. Платой за это становится равномерность: при малом числе виртуальных узлов кольцо даёт перекося (39 % на одном шарде при 100 узлах), который уходит при сотнях узлов. Для TaskManager выбран consistent hashing с 500 виртуальными узлами: сервис растёт, шарды придётся добавлять, и перенос четверти данных вместо трёх четвертей определяет длительность окна миграции, а перекося около 2 % на такой конфигурации несущественен. Настоящий перенос на четвёртый PostgreSQL совпал с расчётом до задачи — 273 461 задача за 8 секунд.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 PostgreSQL 16 Documentation. COPY [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/sql-copy.html> (дата обращения: 15.09.2026).
- 2 PostgreSQL 16 Documentation. 25.1. Routine Vacuuming [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/routine-vacuuming.html> (дата обращения: 15.09.2026).
- 3 Npgsql Documentation. COPY [Электронный ресурс]. – URL: <https://www.npgsql.org/doc/copy.html> (дата обращения: 15.09.2026).
- 4 Karger D., Lehman E., Leighton T. [et al.] Consistent hashing and random trees: distributed caching protocols for relieving hot spots on the World Wide Web // Proceedings of the 29th Annual ACM Symposium on Theory of Computing. – 1997. – P. 654–663.
- 5 DeCandia G., Hastorun D., Jampani M. [et al.] Dynamo: Amazon's highly available key-value store // Proceedings of the 21st ACM Symposium on Operating Systems Principles. – 2007. – P. 205–220.
- 6 Клеппман М. Высоконагруженные приложения. Программирование, масштабирование, поддержка. – СПб. : Питер, 2018. – 640 с.
- 7 Лабораторная работа № 5. Шардирование PostgreSQL: Router и Consistent Hashing : методические указания. – Сириус : Колледж Университета «Сириус», 2026. – 5 с.
- 8 VHS — инструмент записи терминала [Электронный ресурс]. – URL: <https://github.com/charmbracelet/vhs> (дата обращения: 15.09.2026).

ПРИЛОЖЕНИЕ А

Router: hash(key) % N и hash ring

Интерфейс `IShardRouter`, стратегии `ModuloShardRouter` и `ConsistentHashRouter` с виртуальными узлами (разделы 3 и 6).

Листинг А.1 – `ShardRouters.cs`

```
using System.Text.Json.Serialization;

namespace TaskManager.Application.Sharding;

/// <summary>
/// How a shard key is mapped to a shard.
/// </summary>
[JsonConverter(typeof(JsonStringEnumConverter))]
public enum ShardStrategy
{
    /// <summary>
    /// shard = hash(key) % N.
    /// </summary>
    Modulo,

    /// <summary>
    /// The first shard point clockwise from hash(key) on a hash ring.
    /// </summary>
    ConsistentHashing
}

/// <summary>
/// Decides which PostgreSQL instance stores the records of a shard key.
/// </summary>
public interface IShardRouter
{
    /// <summary>
    /// Gets the strategy of the router.
    /// </summary>
    ShardStrategy Strategy { get; }

    /// <summary>
    /// Gets the number of shards, numbered from 0.
    /// </summary>
    int ShardCount { get; }

    /// <summary>
    /// Gets the shard of the key.
    /// </summary>
    int GetShard(Guid key);
}

/// <summary>
/// shard = hash(key) % N. Simple and even, but changing N remaps most keys.
/// </summary>
public sealed class ModuloShardRouter : IShardRouter
{
    /// <summary>
    /// Initializes a new instance of the <see cref="ModuloShardRouter"/> class.
    /// </summary>
    public ModuloShardRouter(int shardCount)
    {
        ArgumentOutOfRangeException.ThrowIfLessThan(shardCount, 1);
        ShardCount = shardCount;
    }

    /// <inheritdoc/>
    public ShardStrategy Strategy => ShardStrategy.Modulo;
}
```

```

    /// <inheritdoc/>
    public int ShardCount { get; }

    /// <inheritdoc/>
    public int GetShard(Guid key) => (int)(ShardHash.Of(key) % (ulong)ShardCount);
}

/// <summary>
/// Consistent hashing: every shard owns <c>virtualNodes</c> points on a ring of 2^64
positions,
/// a key belongs to the first point clockwise from its hash. Adding shard N only adds its
points,
/// so the only keys that move are those that now land on the new shard.
/// </summary>
public sealed class ConsistentHashRouter : IShardRouter
{
    private readonly ulong[] _points;
    private readonly int[] _owners;

    /// <summary>
    /// Initializes a new instance of the <see cref="ConsistentHashRouter"/> class.
    /// </summary>
    /// <param name="shardCount">The number of shards.</param>
    /// <param name="virtualNodes">Points per shard on the ring; more points give a more even
split.</param>
    public ConsistentHashRouter(int shardCount, int virtualNodes = 100)
    {
        ArgumentOutOfRangeException.ThrowIfLessThan(shardCount, 1);
        ArgumentOutOfRangeException.ThrowIfLessThan(virtualNodes, 1);
        ShardCount = shardCount;
        VirtualNodes = virtualNodes;

        // The point of a virtual node depends only on its shard and number, not on N:
        // the ring for N + 1 shards is the ring for N shards plus the points of the new shard
        var ring = new List<(ulong Point, int Shard)>(shardCount * virtualNodes);
        for (var shard = 0; shard < shardCount; shard++)
        {
            for (var node = 0; node < virtualNodes; node++)
            {
                ring.Add((ShardHash.Of($"shard-{shard}#vnode-{node}"), shard));
            }
        }

        ring.Sort((a, b) => a.Point.CompareTo(b.Point));
        _points = ring.Select(r => r.Point).ToArray();
        _owners = ring.Select(r => r.Shard).ToArray();
    }

    /// <inheritdoc/>
    public ShardStrategy Strategy => ShardStrategy.ConsistentHashing;

    /// <inheritdoc/>
    public int ShardCount { get; }

    /// <summary>
    /// Gets the number of points each shard owns on the ring.
    /// </summary>
    public int VirtualNodes { get; }

    /// <inheritdoc/>
    public int GetShard(Guid key)
    {
        var index = Array.BinarySearch(_points, ShardHash.Of(key));
        if (index < 0)
        {
            // Not an exact hit: ~index is the first point greater than the hash
            index = ~index;
        }

        // Past the last point the ring wraps around to the first one

```

```

        return _owners[index == _points.Length ? 0 : index];
    }
}

/// <summary>
/// Creates routers from settings.
/// </summary>
public static class ShardRouterFactory
{
    /// <summary>
    /// Creates a router of the given strategy.
    /// </summary>
    public static IShardRouter Create(ShardStrategy strategy, int shardCount, int virtualNodes)
=> strategy switch
    {
        ShardStrategy.Modulo => new ModuloShardRouter(shardCount),
        _ => new ConsistentHashRouter(shardCount, virtualNodes)
    };
}

```

ПРИЛОЖЕНИЕ Б

Расчёт распределения и переноса данных

Класс `ShardPlanner`: распределение ключей по шардам и подсчёт записей, меняющих шард при изменении их количества (разделы 4, 5 и 7).

Листинг Б.1 – `ShardPlanner.cs`

```
namespace TaskManager.Application.Sharding;

/// <summary>
/// Pure calculations over shard keys and their record counts: distribution and data movement.
/// </summary>
public static class ShardPlanner
{
    /// <summary>
    /// Distributes keys over shards with the router and measures how even the result is.
    /// </summary>
    /// <param name="recordsByKey">Records per shard key.</param>
    /// <param name="router">The router.</param>
    public static ShardDistribution Distribute(IReadOnlyDictionary<Guid, long> recordsByKey,
        IShardRouter router)
    {
        var keys = new long[router.ShardCount];
        var records = new long[router.ShardCount];
        foreach (var (key, count) in recordsByKey)
        {
            var shard = router.GetShard(key);
            keys[shard]++;
            records[shard] += count;
        }

        return Describe(router, keys, records);
    }

    /// <summary>
    /// Compares the shard of every key before and after the change of routers.
    /// </summary>
    /// <param name="recordsByKey">Records per shard key.</param>
    /// <param name="before">The router before the change.</param>
    /// <param name="after">The router after the change.</param>
    public static RebalancePlan Plan(
        IReadOnlyDictionary<Guid, long> recordsByKey, IShardRouter before, IShardRouter after)
    {
        var plan = new RebalancePlan
        {
            Strategy = after.Strategy,
            VirtualNodes = (after as ConsistentHashRouter)?.VirtualNodes,
            FromShards = before.ShardCount,
            ToShards = after.ShardCount
        };

        var moves = new SortedDictionary<(int From, int To), long>();
        foreach (var (key, count) in recordsByKey)
        {
            plan.TotalKeys++;
            plan.TotalRecords += count;

            var from = before.GetShard(key);
            var to = after.GetShard(key);
            if (from == to)
            {
                continue;
            }

            plan.MovedKeys++;
            plan.MovedRecords += count;
            moves[(from, to)] = moves.GetValueOrDefault((from, to)) + count;
        }
    }
}
```

```

    }

    plan.MovedPercent = plan.TotalRecords == 0 ? 0 : Math.Round(100.0 * plan.MovedRecords /
plan.TotalRecords, 2);
    plan.Moves = moves.ToDictionary(m => $"{m.Key.From} → {m.Key.To}", m => m.Value);
    plan.After = Distribute(recordsByKey, after);
    return plan;
}

/// <summary>
/// Builds the distribution summary from per-shard counters.
/// </summary>
public static ShardDistribution Describe(IShardRouter router, IReadOnlyList<long> keys,
IReadOnlyList<long> records)
{
    var total = records.Sum();
    var average = (double)total / records.Count;
    var deviation = Math.Sqrt(records.Sum(r => Math.Pow(r - average, 2)) / records.Count);

    return new ShardDistribution
    {
        Strategy = router.Strategy,
        ShardCount = router.ShardCount,
        VirtualNodes = (router as ConsistentHashRouter)?.VirtualNodes,
        Shards = records
            .Select((r, i) => new ShardLoad(i, keys[i], r, total == 0 ? 0 :
Math.Round(100.0 * r / total, 2)))
            .ToList(),
        MaxToAverage = total == 0 ? 0 : Math.Round(records.Max() / average, 3),
        DeviationPercent = total == 0 ? 0 : Math.Round(100 * deviation / average, 2)
    };
}
}

```

ПРИЛОЖЕНИЕ В

Загрузка и перенос данных между шардами

Часть класса `ShardedTaskStore`: загрузка задач основной базы на шарды через COPY и перенос пользователей с шарда на шард (раздел 8).

Листинг В.1 – `ShardedTaskStore.DataMovement.cs`

```
using Microsoft.Extensions.Logging;
using Npgsql;
using NpgsqlTypes;
using TaskManager.Application.Sharding;

namespace TaskManager.Infrastructure.Repositories;

/// <summary>
/// Bulk data movement: initial load from the main database and moving keys between shards.
/// </summary>
public partial class ShardedTaskStore
{
    private const int MoveBatchKeys = 500;

    /// <inheritdoc/>
    public async Task<long> LoadFromMainDatabaseAsync(IShardRouter router, CancellationToken
cancellationToken = default)
    {
        if (router.ShardCount > ConfiguredShards)
        {
            throw new InvalidOperationException($"{router.ShardCount} shards requested,
{ConfiguredShards} configured");
        }

        // Earlier loads may have used more shards: clear every shard that answers
        for (var shard = 0; shard < ConfiguredShards; shard++)
        {
            try
            {
                await using var connection = await OpenShardAsync(shard, cancellationToken);
                await using var truncate = new NpgsqlCommand("TRUNCATE tasks", connection);
                await truncate.ExecuteNonQueryAsync(cancellationToken);
            }
            catch (Exception ex) when (shard >= router.ShardCount && ex is NpgsqlException or
System.Net.Sockets.SocketException)
            {
                _logger.LogInformation("Shard {Shard} is not running and is not used, skipped",
shard);
            }
        }

        var targets = new NpgsqlConnection[router.ShardCount];
        var writers = new NpgsqlBinaryImporter[router.ShardCount];
        try
        {
            for (var shard = 0; shard < router.ShardCount; shard++)
            {
                targets[shard] = await OpenShardAsync(shard, cancellationToken);
                writers[shard] = await targets[shard].BeginBinaryImportAsync(
                    $"COPY tasks ({Columns}) FROM STDIN (FORMAT BINARY)", cancellationToken);
            }

            // One streaming pass over the main table; every row goes to the COPY stream of its
shard
            await using var source = new NpgsqlConnection(_mainConnectionString);
            await source.OpenAsync(cancellationToken);
            await using var select = new NpgsqlCommand($"SELECT {Columns} FROM tasks", source);
            await using var reader = await select.ExecuteReaderAsync(cancellationToken);
```

```

        long loaded = 0;
        while (await reader.ReadAsync(cancellationToken))
        {
            var userId = reader.GetGuid(1);
            await CopyRowAsync(reader, writers[router.GetShard(userId)],
cancellationToken);
            loaded++;
        }

        foreach (var writer in writers)
        {
            await writer.CompleteAsync(cancellationToken);
        }

        logger.LogInformation("Loaded {Count} tasks to {Shards} shards with {Strategy}",
loaded, router.ShardCount, router.Strategy);
        return loaded;
    }
    finally
    {
        foreach (var writer in writers.Where(w => w is not null))
        {
            await writer.DisposeAsync();
        }

        foreach (var target in targets.Where(t => t is not null))
        {
            await using var analyze = new NpgsqlCommand("ANALYZE tasks", target);
            await analyze.ExecuteNonQueryAsync(CancellationTokens.None);
            await target.DisposeAsync();
        }
    }
}

/// <inheritdoc/>
public async Task<long> MoveKeysAsync(
    int fromShard, int toShard, IReadOnlyCollection<Guid> keys, CancellationTokens
cancellationToken = default)
{
    long moved = 0;
    foreach (var batch in keys.Chunk(MoveBatchKeys))
    {
        await using var source = await OpenShardAsync(fromShard, cancellationToken);
        await using var target = await OpenShardAsync(toShard, cancellationToken);

        // 1. Copy into a temporary table of the target and insert what is not there yet:
        // if a previous attempt stopped after the copy, those rows are simply skipped
        await using (var transaction = await
target.BeginTransactionAsync(cancellationToken))
        {
            await using (var create = new NpgsqlCommand(
                "CREATE TEMP TABLE incoming (LIKE tasks) ON COMMIT DROP",
target, transaction))
            {
                await create.ExecuteNonQueryAsync(cancellationToken);

                await using (var writer = await target.BeginBinaryImportAsync(
                    $"COPY incoming ({Columns}) FROM STDIN (FORMAT BINARY)",
cancellationToken))
                {
                    await using var select = new NpgsqlCommand($"SELECT {Columns} FROM tasks
WHERE user_id = ANY(@keys)", source);
                    select.Parameters.AddWithValue("keys", batch);
                    await using var reader = await
select.ExecuteReaderAsync(cancellationToken);
                    while (await reader.ReadAsync(cancellationToken))
                    {
                        await CopyRowAsync(reader, writer, cancellationToken);
                    }

                    await writer.CompleteAsync(cancellationToken);
                }
            }
        }
    }
}

```

```

        await using (var insert = new NpgsqlCommand(
            "INSERT INTO tasks SELECT * FROM incoming ON CONFLICT DO
NOTHING", target, transaction))
        {
            await insert.ExecuteNonQueryAsync(cancellationToken);
        }

        await transaction.CommitAsync(cancellationToken);
    }

    // 2. Rows are deleted from the source only after the copy is committed. Two
databases
    // cannot share one transaction, so a failure between the steps leaves a
duplicate, never a loss
    await using var delete = new NpgsqlCommand("DELETE FROM tasks WHERE user id =
ANY(@keys)", source);
    delete.Parameters.AddWithValue("keys", batch);
    moved += await delete.ExecuteNonQueryAsync(cancellationToken);
}

return moved;
}

private static async Task CopyRowAsync(NpgsqlDataReader reader, NpgsqlBinaryImporter
writer, CancellationToken cancellationToken)
{
    await writer.StartRowAsync(cancellationToken);
    await writer.WriteAsync(reader.GetGuid(0), NpgsqlDbType.Uuid, cancellationToken);
    await writer.WriteAsync(reader.GetGuid(1), NpgsqlDbType.Uuid, cancellationToken);
    await WriteNullableAsync(writer, reader.IsDBNull(2) ? (Guid?)null : reader.GetGuid(2),
NpgsqlDbType.Uuid, cancellationToken);
    await writer.WriteAsync(reader.GetString(3), NpgsqlDbType.Varchar, cancellationToken);
    await WriteNullableAsync(writer, reader.IsDBNull(4) ? null : reader.GetString(4),
NpgsqlDbType.Text, cancellationToken);
    await writer.WriteAsync(reader.GetInt32(5), NpgsqlDbType.Integer, cancellationToken);
    await writer.WriteAsync(reader.GetInt32(6), NpgsqlDbType.Integer, cancellationToken);
    await WriteNullableAsync(writer, reader.IsDBNull(7) ? (DateTime?)null :
reader.GetFieldValue<DateTime>(7), NpgsqlDbType.TimestampTz, cancellationToken);
    await writer.WriteAsync(reader.GetFieldValue<DateTime>(8), NpgsqlDbType.TimestampTz,
cancellationToken);
    await writer.WriteAsync(reader.GetFieldValue<DateTime>(9), NpgsqlDbType.TimestampTz,
cancellationToken);
}

private static async Task WriteNullableAsync<T>(
    NpgsqlBinaryImporter writer, T? value, NpgsqlDbType type, CancellationToken
cancellationToken)
{
    if (value is null)
    {
        await writer.WriteNullAsync(cancellationToken);
    }
    else
    {
        await writer.WriteAsync(value, type, cancellationToken);
    }
}
}

```